

Informe de flujo SNN a FPGA-in-the-Loop

HDL Coder y HDL Verifier para MATLAB/Simulink

Ejemplo de red neuronal espiking

Documento técnico

Proyecto SNN

Responsable: Autor de Referencia

3 de enero de 2026

Índice general

1. Introducción	1
1.1. Introducción a las Spiking Neural Networks	1
1.1.1. Funcionamiento básico	1
1.1.2. Ventajas de las SNN.....	1
1.1.3. Aplicaciones principales	1
1.1.4. Conclusión.....	2
1.2. Requisitos previos	2
1.3. Flujo general	2
2. Modelado de la SNN en MATLAB/Simulink	3
2.1. Modelo matemático básico (sin multiplicaciones).....	3
2.1.1. Entrada sináptica (suma condicionada)	3
2.1.2. Dinámica y disparo.....	3
2.1.3. Definición de variables	3
2.2. Enfoque de la prueba: SNN simple para MNIST	4
2.3. Entrenamiento en Python y artefactos generados	4
2.4. Implementación de la SNN en MATLAB	4
2.4.1. Parámetros y pesos en <code>snn_params.m</code>	5
2.4.2. Script de generación de parámetros	6
2.5. Banco de pruebas MATLAB con entrada empaquetada	6
3. Generación de HDL con HDL Coder	8
3.1. Flujo con HDL Coder (Workflow Advisor)	8
3.2. Conversión a punto fijo	8
3.3. Configuración de generación HDL	9
3.3.1. Optimización y uso de recursos	10
3.4. Salida de generación y lectura de logs	11
4. Verificación	13
4.1. Verificación con Workflow Advisor	13
4.1.1. Co-simulación (HDL Verifier)	13
4.1.2. Prueba en FPGA-in-the-Loop.....	14

4.2.	Verificación en Simulink.....	15
4.2.1.	Integración con bloque FIL	15
4.2.2.	Prueba con Computer Vision Toolbox	17
5.	Resultados y conclusiones	19
5.1.	Resultados y hallazgos	19
5.2.	Conclusiones.....	19
5.3.	Trabajo futuro.....	19

Índice de figuras

3.1	Carga del DUT y testbench en Workflow Advisor.	8
3.2	Configuraciones clave en <i>Fixed-Point Conversion</i>	9
3.3	Comparación de salidas: flotante vs fijo.	9
3.4	Configuración de generación y optimización (a) target de generación, (b) optimización con <i>Stream Loop</i> , (c) <i>Clocks & Ports</i>	10
3.5	Impacto de la optimización en el uso de recursos.	11
4.1	Configuración de co-simulación.....	13
4.2	Resultado de co-simulación.....	14
4.3	Configuración de FIL.....	14
4.4	Resultado de FIL.....	15
4.5	Inserción y configuración del bloque FIL.....	16
4.6	Captura y análisis en Simulink.....	16
4.7	Diagrama de comparación con retardo de 1 ciclo.	17
4.8	Resultado comparado MATLAB vs FIL.	17
4.9	Prueba con Computer Vision Toolbox.	18

1. Introducción

1.1. Introducción a las Spiking Neural Networks

Las **Spiking Neural Networks (SNN)** son redes neuronales inspiradas en el cerebro biológico. Procesan información mediante **spikes**, eventos discretos en el tiempo que activan el cómputo sólo cuando ocurren. Esto produce un modelo **asíncrono y basado en eventos**, más cercano a sistemas neuronales reales.

1.1.1. Funcionamiento básico

Cada neurona integra los spikes entrantes sumando los pesos sinápticos. Si el potencial de membrana supera un umbral, la neurona emite un spike y se reinicia. Un modelo típico es la neurona **Leaky Integrate-and-Fire (LIF)**. La ausencia de spike implica ausencia de cómputo, permitiendo implementar el modelo sin multiplicaciones explícitas, solo con sumas condicionadas.

1.1.2. Ventajas de las SNN

- Alta eficiencia energética al operar solo cuando hay eventos.
- Procesamiento temporal nativo, ideal para señales dependientes del tiempo.
- Computación asíncrona, sin necesidad de reloj global.
- Mayor plausibilidad biológica frente a ANN y DNN.
- Implementación eficiente en hardware neuromórfico.

1.1.3. Aplicaciones principales

Las SNN son adecuadas cuando el tiempo y el consumo energético son críticos:

- Visión por eventos (cámaras neuromórficas).
- Robótica autónoma para percepción y control en tiempo real.
- Procesamiento de señales temporales (audio, biosenales).
- Sistemas embebidos de bajo consumo.
- Interfaces cerebro-máquina y neuroingeniería.

1.1.4. Conclusión

Las SNN ofrecen un enfoque eficiente y temporalmente expresivo para procesar información. Su ventaja energética y compatibilidad con hardware neuromórfico las hace prometedoras para aplicaciones en tiempo real y bajo consumo.

1.2. Requisitos previos

- Herramientas: MATLAB/Simulink con HDL Coder y HDL Verifier, soporte de placa objetivo.
- Versiones recomendadas y licencias activas.
- Archivos de partida del modelo SNN y entorno de simulación configurado.

1.3. Flujo general

1. Modelado y validación de la SNN en MATLAB/Simulink.
2. Generación de HDL con HDL Coder.
3. Integración y pruebas FPGA-in-the-Loop con HDL Verifier (FIL).

2. Modelado de la SNN en MATLAB/Simulink

2.1. Modelo matemático básico (sin multiplicaciones)

Sea $s_i(t) \in \{0, 1\}$ el spike de la sinapsis i en el tiempo t , y w_i su peso.

2.1.1. Entrada sináptica (suma condicionada)

$$I(t) = \sum_{i=1}^N \begin{cases} w_i & \text{si } s_i(t) = 1 \\ 0 & \text{si } s_i(t) = 0 \end{cases}$$

Equivalente:

$$I(t) = \sum_{i: s_i(t)=1} w_i$$

2.1.2. Dinámica y disparo

$$V(t) = V(t-1) + I(t)$$

$$y(t) = \begin{cases} 1 & \text{si } V(t) \geq \theta \\ 0 & \text{si } V(t) < \theta \end{cases}$$

Solo se suman los pesos de las entradas que generan spike.

2.1.3. Definición de variables

- $s_i(t)$: spike de la neurona presináptica i en el instante t (1 si hay spike, 0 si no).
- w_i : peso sináptico asociado a la conexión i , define la contribución de un spike.
- $I(t)$: entrada sináptica total en el tiempo t , suma de los pesos de sinapsis que dispararon.
- $V(t)$: potencial de membrana de la neurona en el instante t , acumula contribuciones entrantes.
- θ : umbral de disparo, potencial mínimo para generar un spike.
- $y(t)$: spike de salida en el tiempo t (1 si dispara, 0 en caso contrario).

2.2. Enfoque de la prueba: SNN simple para MNIST

Para esta prueba se usará un ejemplo sencillo: creación de un core SNN orientado a clasificar imágenes de MNIST. El conjunto MNIST contiene imágenes en blanco y negro de 28×28 píxeles de dígitos manuscritos (0 al 9).

La arquitectura será mínima para facilitar la implementación posterior en hardware:

- 784 neuronas de entrada, una por píxel.
- 10 neuronas de salida, una por clase (dígitos 0–9).

El objetivo no es maximizar rendimiento, sino validar el flujo completo de diseño: definir el modelo, entrenarlo y generar un core en HDL. El entrenamiento del modelo se realizará en Python, aprovechando librerías de SNN; el foco será la implementación del core en HDL generada desde MATLAB, obteniendo un diseño sintetizable y verificable en hardware digital.

2.3. Entrenamiento en Python y artefactos generados

Para implementar la SNN planteada, se entrenó en Python utilizando codificación de latencia. Se usa un solo paso temporal y una salida directa desde la entrada para simplificar la explicación y la visualización.

- Matriz de pesos entrenada: `fc1_full_weights.csv`.
- 100 frames de spikes de prueba en `spike_train.mem`, representativos de imágenes de 28×28 .
- 100 imágenes de entrada en la carpeta `frames/`.
- Resultados esperados de los frames de prueba (etiquetas y salidas).

2.4. Implementación de la SNN en MATLAB

La implementación principal está en `snn.m`. El núcleo recorre cada neurona de salida, acumula únicamente los pesos cuyos spikes de entrada están activos y compara contra un umbral común.

```
function spikes_out = snn(spikes_in)
    % SNN  Computa la salida de una red neuronal de disparos binaria.
```

```

params = snn_params();
W = params.W;           % Matriz de pesos MxN (M entradas, N salidas)
thresh = params.thresh; % Umbral de disparo para cada neurona de salida

[M, N] = size(W);       % M=784, N=10 (por ejemplo)

% Inicializa vector de salida como ufixN (en este caso ufix10)
spikes_out = fi(0, 0, N, 0);

% Recorre cada neurona de salida j
for j = 1:N
    acc = 0;             % Acumulador de la suma ponderada de entradas
    % Suma W(i,j) solo si la neurona de entrada i está disparando
    for i = 1:M
        % bit 1 = LSB, y que i=1 corresponde al MSB:
        %   - i=1 --> bit M
        %   - i=M --> bit 1
        if bitget(spikes_in, M - i + 1)
            acc = acc + W(i,j);
        end
    end
    % Si la suma supera el umbral, activa el bit correspondiente en la salida
    % j=1 como MSB de la salida
    if acc >= thresh
        spikes_out = bitset(spikes_out, N - j + 1, 1);
    end
end
end
end

```

2.4.1. Parámetros y pesos en snn_params.m

La función `snn_params()` contiene los parámetros de la red neuronal. El archivo es autogenerado (por `generate_hdl.m`) e incluye la matriz de pesos completa de 784×10 y el umbral de disparo. A continuación se muestra un fragmento representativo:

```

function params = snn_params()
% Autogenerado por generate_hdl.m
params.thresh = 1 ;
params.beta = 1;
params.W = [ ...
    -0.14369439 0.00000000 -0.00000000 -0.10920773 -0.00000000 ...
    -0.10345996 0.00000000 0.00000000 -0.11495551 -0.29313657 ...
    0.00000000 -0.13219884 0.14944217 -0.14369439 -0.30463210 ...
    -0.08621664 -0.00000000 0.00000000 -0.00000000 -0.26439768 ...
    -0.00000000 -0.00000000 -0.12070329 -0.00000000 -0.19542436 ...

```

...]

2.4.2. Script de generación de parámetros

El archivo de parámetros se construye a partir del umbral y la matriz de pesos entrenada. El siguiente script escribe el archivo de parámetros para MATLAB:

```
function write_params_file(filename, tresh, beta)
    fid = fopen(filename, 'w');

    fprintf(fid, 'function params = snn_params()\n');
    fprintf(fid, '%% Autogenerado por generate_hdl.m\n');
    fprintf(fid, 'params.tresh = %d;\n', params.FRAC);
    fprintf(fid, 'params.beta = %d;\n', params.WL_W);
    for r = 1:size(params.W, 1)
        fprintf(fid, ' %s;\n', row_as_text(params.W(r, :)));
    end
    fprintf(fid, '];\n');
    fprintf(fid, 'end\n');
    fclose(fid);
end
```

2.5. Banco de pruebas MATLAB con entrada empaquetada

En FPGA-in-the-Loop, los puertos vectoriales ($N \times 1$) no están soportados y HDL Coder los *scalariza*, lo que convierte un vector de 784 elementos en 784 señales escalares y termina exponiendo 784 puertos en el bloque FIL. Para lograr un solo puerto de 784 bits, se usa un escalar con *word length* de 784 bits (bus empaquetado). En Simulink/MATLAB esto se representa como un tipo fijo sin fracción (`ufix784`). HDL Coder permite longitudes de palabra grandes (hasta 65535 bits), lo que habilita este empaquetado en FIL. El banco de pruebas toma los frames de `spike_train.mem`, los empaqueta en `ufix784` y ejecuta la SNN frame a frame. La conversión respeta el mapeo MSB/LSB para mantener la correspondencia con el vector de 784 entradas.

```
mem_file = 'spike_train.mem';

%% --- CARGAR FRAMES ---
fid = fopen(mem_file, 'r');
if fid < 0, error('No pude abrir %s', mem_file); end
C = textscan(fid, '%s', 'Delimiter', '\n');
fclose(fid);
```

```

frames = C{1};

%% --- CREAR MATRIZ LOGICAL DE FRAMES ---
char_mat = char(strtrim(frames)); % cada fila es "0101..."
frames_mat = (char_mat == '1'); % logical (num_frames x 784)

%% --- EMPAQUETAR TODOS LOS FRAMES A ufix784 ---
packed_in = fi(zeros(num_frames,1), 0, 784, 0); % num_frames x 1, ufix784

% frames_mat(f,1) -> bit 784 (MSB)
% frames_mat(f,784) -> bit 1 (LSB)
for i = 1:784
    packed_in = bitset(packed_in, 784 - i + 1, frames_mat(:, i));
end
in_ts = timeseries(packed_in, (0:num_frames-1)'.*0.1);

for f = 1:num_frames
    out = snn(packed_in(f)); % ufix10
    fprintf('Frame %d -> %s\n', f, bin(out));
end

```

3. Generación de HDL con HDL Coder

3.1. Flujo con HDL Coder (Workflow Advisor)

El flujo se realizó con *Workflow Advisor*, cargando `snn.m` como DUT y el testbench en MATLAB.

- Selección de *MATLAB Function* como DUT.
- Definición del testbench y de los tipos de entrada/salida.
- Revisión rápida de compatibilidad antes de convertir a punto fijo.

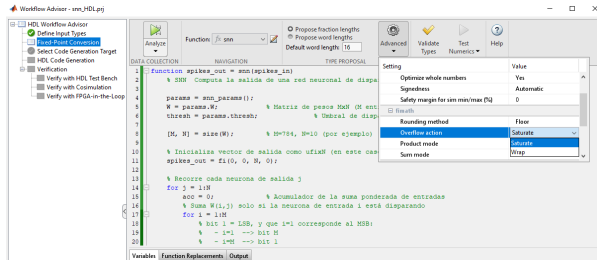


Figura 3.1: Carga del DUT y testbench en Workflow Advisor.

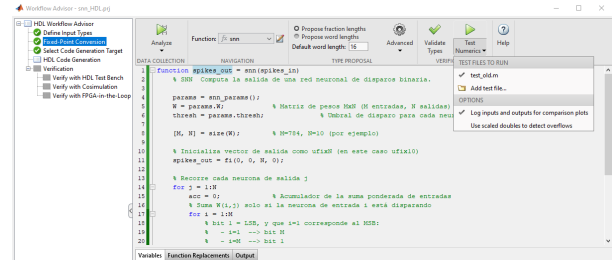
3.2. Conversión a punto fijo

En *MATLAB to HDL* se usa la opción *Convert to fixed point*. Si se desea mantener punto flotante, se debe seleccionar *Keep original type* y habilitar *HDL code generation* para punto flotante.

- En *Type Proposal* se recomienda *Propose fraction lengths*.
- *Default word length* de 16 bits para pesos y acumulación.
- Se usa *Saturate* para evitar overflow/underflow en el acumulador.



(a) Conversión a punto fijo con saturación.



(b) Comparación de entradas/salidas.

Figura 3.2: Configuraciones clave en *Fixed-Point Conversion*.

Luego se ejecuta *Validate Types*. Una vez sin errores, se activa *Configure Test Numerics* para comparar salida del modelo flotante vs fijo. El gráfico resultante muestra que 1 frame de 100 difiere, debido a la reducción de precisión a 16 bits. Para eliminarlo, se incrementa el número de bits.

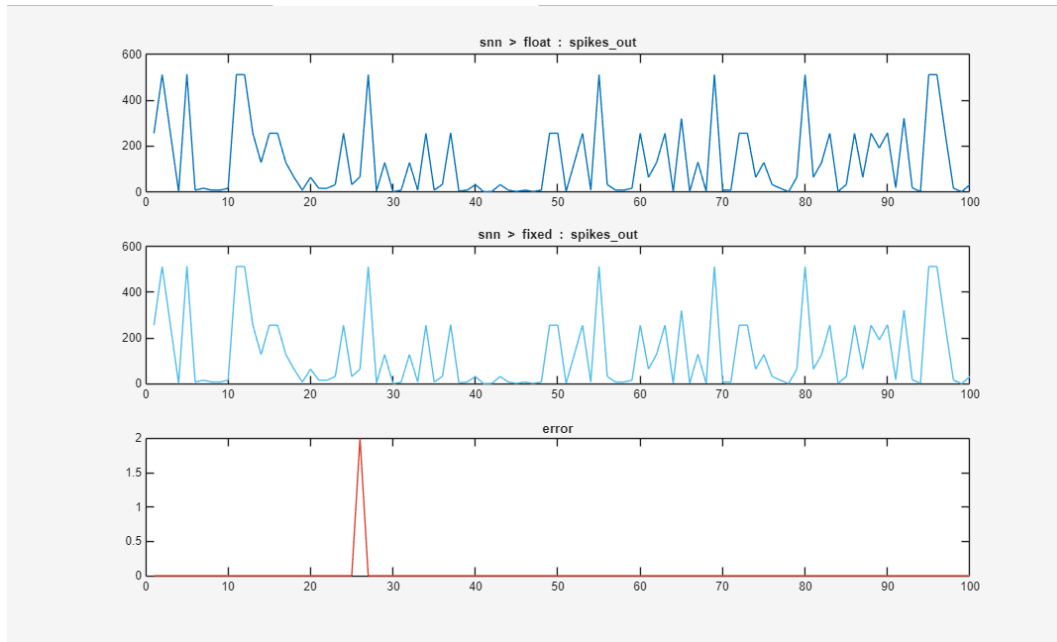


Figura 3.3: Comparación de salidas: flotante vs fijo.

3.3. Configuración de generación HDL

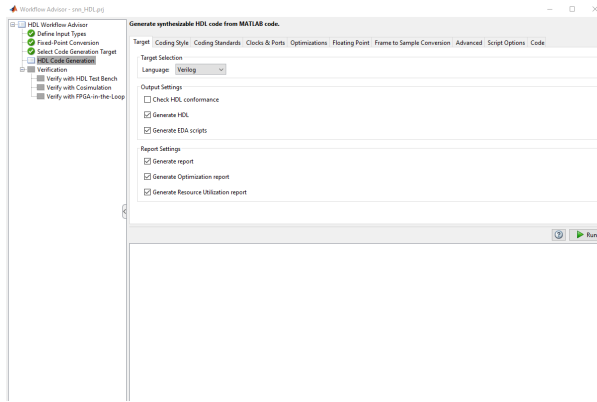
Se recomienda:

- Mantener *Code Generation Target* por defecto.
- Generar HDL en Verilog (con VHDL se observaron fallas en pruebas).
- Activar todos los reportes para tener trazabilidad del proceso.

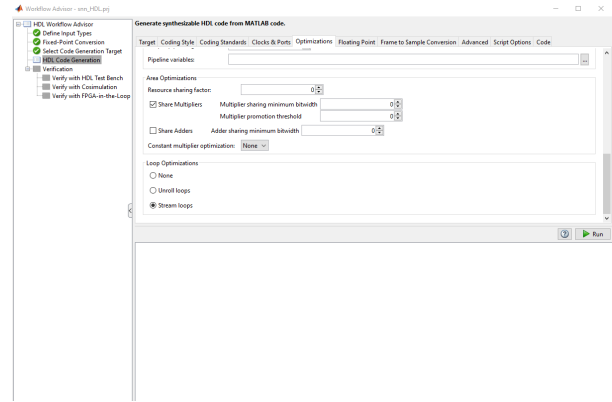
3.3.1. Optimización y uso de recursos

En *Optimization* se configuró *Stream Loop*. Esta opción comparte recursos entre iteraciones y reduce el número de multiplicadores/sumadores, a costa de más ciclos de reloj.

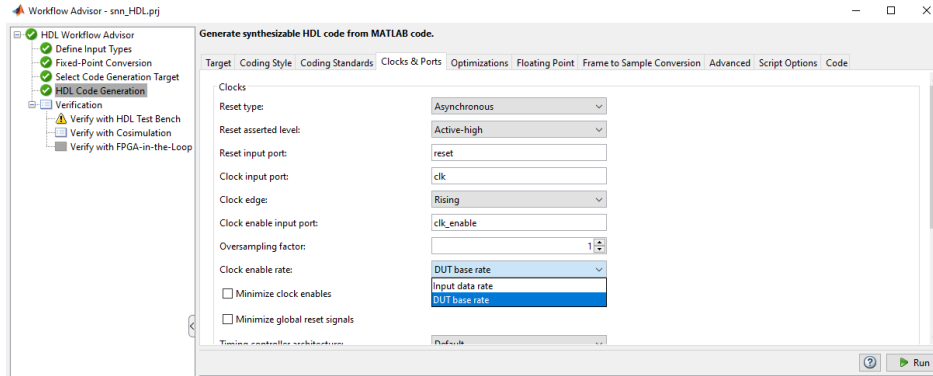
- *None* crea recursos dedicados por iteración (unrolled).
- *Stream Loop* usa un mismo multiplicador y sumador para los dos **for** anidados.
- Con *Stream Loop*, se esperan 7840 ciclos internos por frame.



(a) Target de generación.



(b) Optimización con *Stream Loop*.



(c) Configuración de *Clocks & Ports*.

Figura 3.4: Configuración de generación y optimización (a) target de generación, (b) optimización con *Stream Loop*, (c) *Clocks & Ports*.

Para co-simulación se activa *DUT base rate as clock enable* en *Clocks & Ports*, asegurando alineación entre frecuencia de entrada y *clock enable*.

La diferencia de recursos se explica por el reparto de hardware: con *Stream Loop* se comparte un mismo multiplicador y sumador entre las iteraciones de los dos bucles, reduciendo recursos

a cambio de más ciclos; en cambio, con *None* (unroll) se instancian recursos dedicados por iteración, lo que acelera la operación pero incrementa mucho el uso de LUTs, DSPs y registros. La diferencia de recursos es clara:

Summary

Multipliers	1
Adders/Subtractors	14
Registers	12
Total Register Bits	275
RAMs	0
Multiplexers	18
I/O Bits	798
Shifters	1

(a) Recursos con *Stream Loop*.

Summary

Multipliers	7840
Adders/Subtractors	54910
Registers	0
Total Register Bits	0
RAMs	0
Multiplexers	7840
I/O Bits	794
Shifters	10

(b) Recursos sin *Stream Loop*.

Figura 3.5: Impacto de la optimización en el uso de recursos.

3.4. Salida de generación y lectura de logs

El proceso de generación produce un log de consola. Los mensajes clave son la latencia de 1 ciclo y la necesidad de un reloj $7840\times$ mayor que el *base rate*.

```
### Begin MATLAB to HDL Code Generation...
### Working on DUT: snn_fixpt.
### Using TestBench: test_old.
### The DUT requires an initial pipeline setup latency. Each output port
  ↳ experiences these additional delays.
### Output port 1: 1 cycles.
### MESSAGE: The design requires 7840 times faster clock with respect to the base
  ↳ rate = 1.
### Working on snn_fixpt_tc as snn_fixpt_tc.v.
### Begin Verilog Code Generation
```

```

### Working on snn_fixptp1 as snn_fixptp1.v.
### Working on snn_fixpt_enb_bypass as snn_fixpt_enb_bypass.v.
### Working on snn_fixpt as snn_fixpt.v.
### Generating Resource Utilization Report resource_report.html.
### Generating Optimization report
### To rerun codegen evaluate the following commands...
-----
cgi      =
↳ load('C:\Users\llillo\Desktop\cosas_ordenadas\uni\memoria\git\memoria_fpga_design_matlab\e
cfg      = cgi.CodeGenInfo.codegenSettings;
fxpCfg   = cgi.CodeGenInfo.fxpCfg;
codegen -float2fixed fxpCfg -config cfg -report
-----
### Generating HDL Conformance Report snn_fixpt_hdl_conformance_report.html.
### HDL Conformance check complete with 0 errors, 1 warnings, and 1 messages.
### Code generation successful: View report
### Elapsed Time: '          26.8142' sec(s)

```

Interpretación:

- La salida tiene un retardo fijo de 1 ciclo de *base rate*.
- El diseño requiere un reloj $7840\times$ más rápido debido al uso de recursos compartidos.
- En FIL se debe configurar *oversampling* a 7840.

4. Verificación

4.1. Verificación con Workflow Advisor

La verificación se realiza en dos etapas dentro del flujo de Workflow Advisor: co-simulación y FPGA-in-the-Loop (FIL).

4.1.1. Co-simulación (HDL Verifier)

En la co-simulación se activan todas las opciones para comparar la salida de `snn_fixpt.v` con `snn_fixpt.m` usando el mismo testbench. Esto valida que el HDL genera la misma respuesta que el modelo fijo.

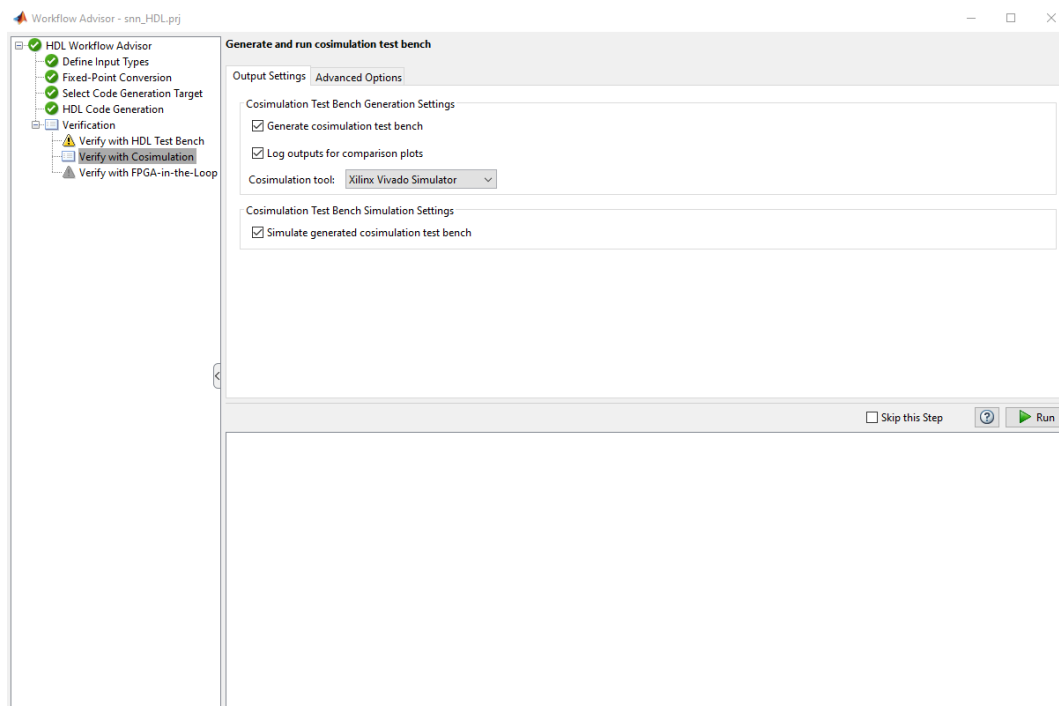


Figura 4.1: Configuración de co-simulación.

Se requiere configurar el simulador en el *path* de MATLAB. Para Vivado:

```
hdlsetuptoolpath('ToolName','Xilinx  
↳ Vivado','ToolPath','C:\Xilinx\Vivado\20xx.x\bin\vivado.bat');
```

El resultado confirma que no hay errores frente a `snn_fixpt.m`. La diferencia de 1 frame que aparece al comparar flotante vs fijo se mantiene.

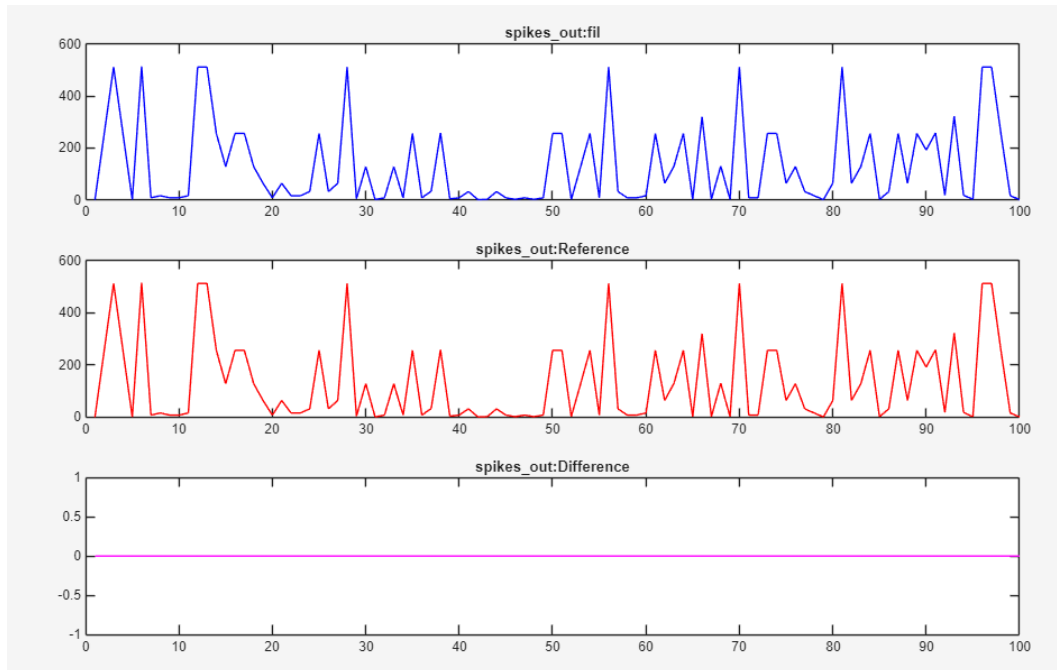


Figura 4.2: Resultado de co-simulación.

4.1.2. Prueba en FPGA-in-the-Loop

En FIL se activan todas las casillas de verificación. El flujo es más lento que la co-simulación porque incluye síntesis, implementación, generación de bitstream y descarga en la FPGA.

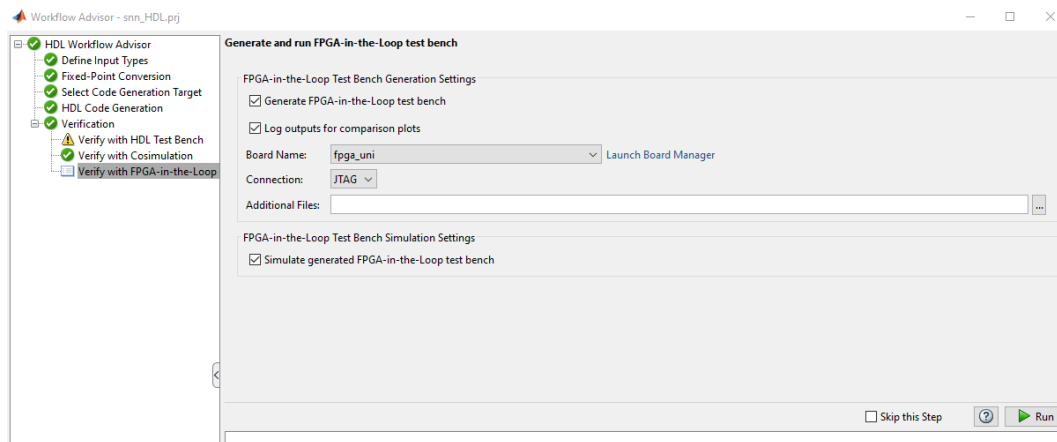


Figura 4.3: Configuración de FIL.

El resultado final coincide con co-simulación:

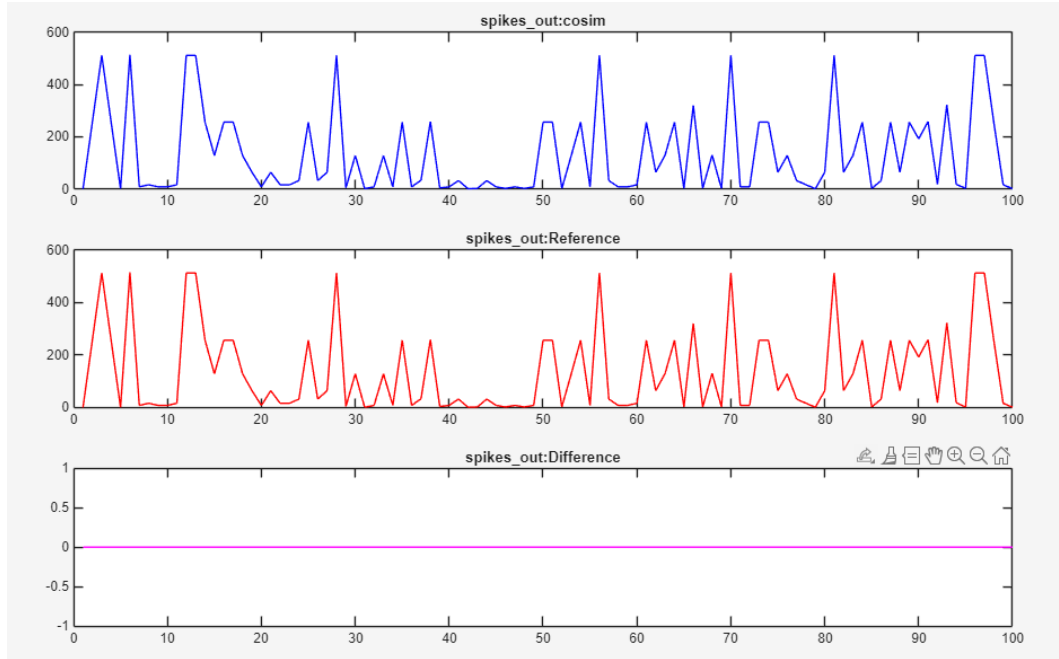


Figura 4.4: Resultado de FIL.

Se recomienda aumentar el número de imágenes de prueba para mayor confianza, ya que el test actual usa 100 frames.

4.2. Verificación en Simulink

Esta etapa replica el flujo de FIL en un entorno de simulación más amigable, integrando el bloque FIL y comparando salidas con el modelo MATLAB.

4.2.1. Integración con bloque FIL

Tras crear el bloque con *FIL Wizard* (ver documento de uso de FIL: TODO: ANIDAR EL DOCUMENTO), se configura:

- *Oversampling* a 7840 por el uso de *Stream Loop*.
- Retardo de 1 ciclo para alinear la salida FIL con la salida de MATLAB.

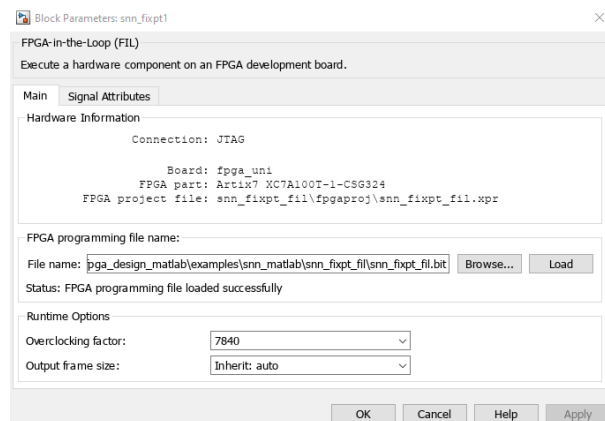
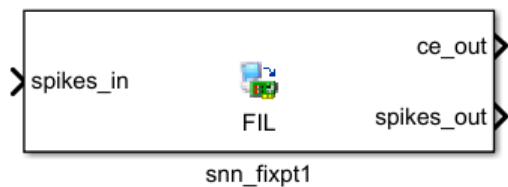


Figura 4.5: Inserción y configuración del bloque FIL.

Durante el test se usa la variable `in_ts` para alimentar el bloque SNN y se activa el *logging* de la señal de salida.

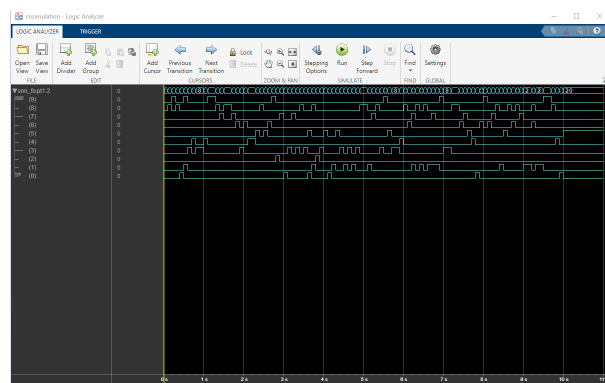
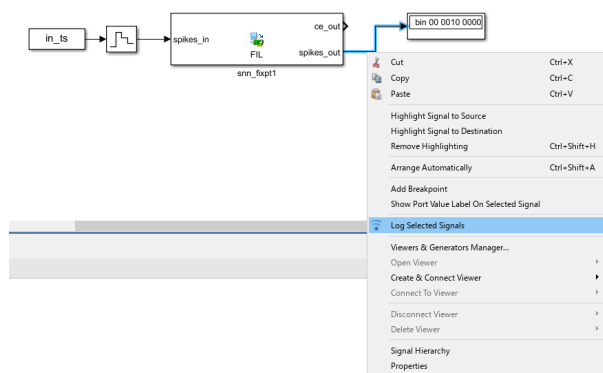


Figura 4.6: Captura y análisis en Simulink.

Para comparar salidas alineadas se agrega un *Unit Delay* de 1 ciclo (0.1 s), debido a la latencia de salida en FIL.

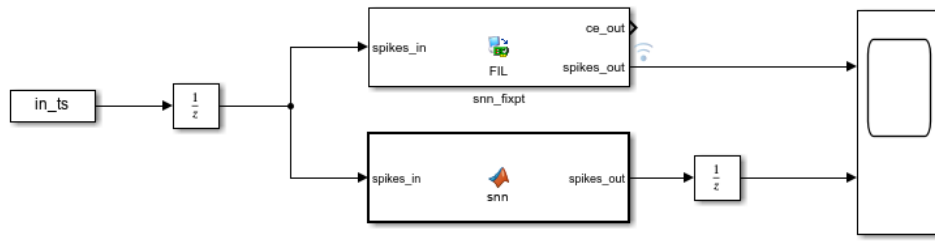


Figura 4.7: Diagrama de comparación con retardo de 1 ciclo.

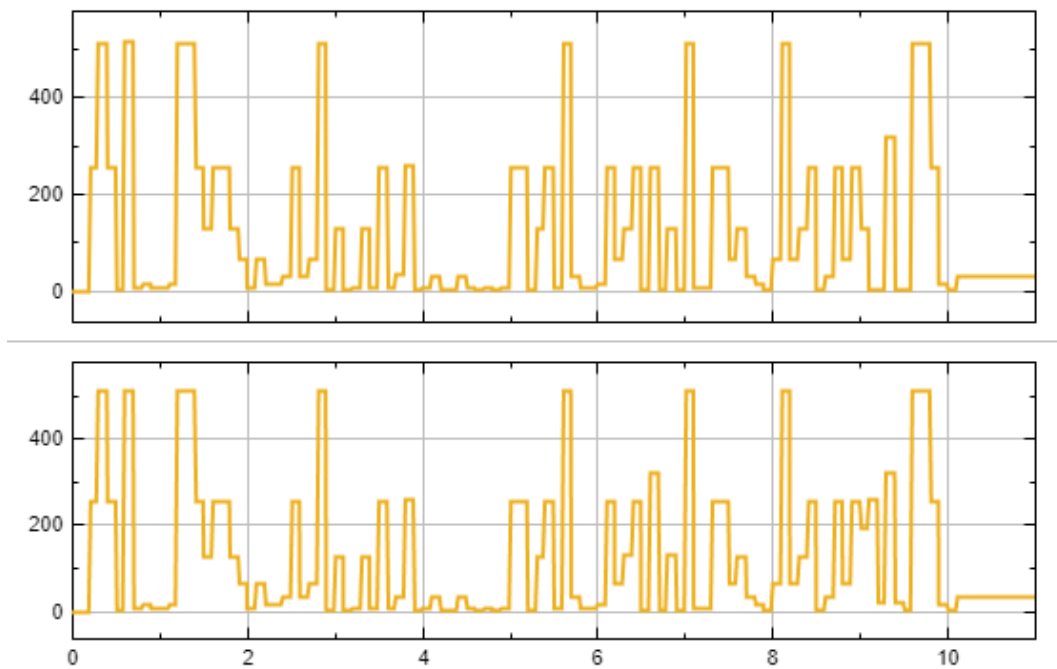


Figura 4.8: Resultado comparado MATLAB vs FIL.

4.2.2. Prueba con Computer Vision Toolbox

Si se dispone de Computer Vision Toolbox, se puede usar *From Image* y *Video Viewer* para ingresar imágenes reales. Se requiere una función que convierta la imagen en frame de spikes:

```
function packed = image_to_packed_latency_ufix784(img, latency_threshold)

    if nargin < 2 || isempty(latency_threshold)
        latency_threshold = 0.5;
    end
```

```

if ~isa(img, 'double')
    img = double(img);
end

if ndims(img) == 3
    if ~isequal(size(img), [28 28 3])
        error('La imagen debe ser 28x28x3.');
```

end

```

    img_gray = rgb2gray(img / max(255, max(img(:)))));
else
    if ~isequal(size(img), [28 28])
        error('La imagen debe ser 28x28.');
```

end

```

    img_gray = img / max(255, max(img(:)));
end

spikes = (img_gray.' > latency_threshold);

packed = fi(0, 0, 784, 0);
for i = 1:784
    packed = bitset(packed, 784 - i + 1, spikes(i));
end
end
end

```

La simulación del frame 044 (dígito 8) enciende la neurona correspondiente, observándose en el display 0000000010 (bit 8 activo).

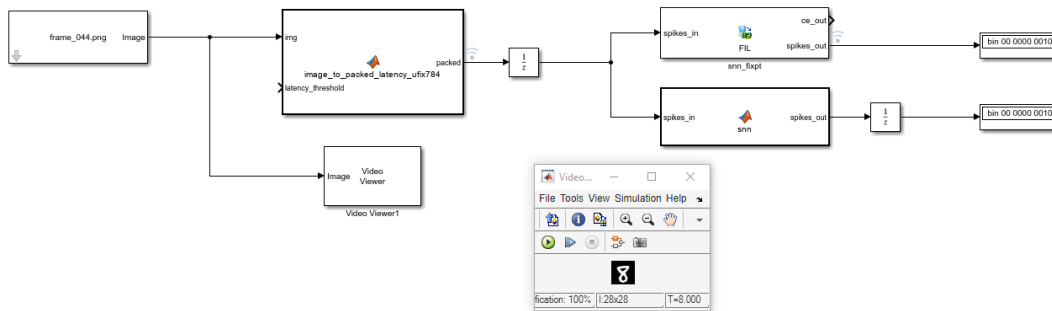


Figura 4.9: Prueba con Computer Vision Toolbox.

5. Resultados y conclusiones

5.1. Resultados y hallazgos

Texto pendiente: resumen de métricas clave y observaciones al validar la SNN en hardware.

5.2. Conclusiones

- Viabilidad del flujo MATLAB → HDL Coder → FIL para la SNN.
- Limitaciones encontradas y riesgos pendientes.
- Recomendaciones para producción.

5.3. Trabajo futuro

- Mejoras de rendimiento o consumo.
- Extensiones del modelo SNN.
- Automatización de pruebas y CI.